

Flutter TS Identity Orchestration - User Guide

Overview

The Flutter TS Identity Orchestration plugin enables secure identity verification and authentication flows in your Flutter app using Transmit Security's Identity Orchestration platform.

Requirements

- **Flutter:** 3.0.0 or higher
- **iOS:** 13.0 or higher
- **Android:** API level 21 (Android 5.0) or higher
- **Dart:** 2.17.0 or higher

Bundled native SDK versions

Each release of this plugin targets a specific set of Transmit native SDK versions. The plugin declares and pins them itself — you do not add, choose, or upgrade the native SDKs, and the plugin is built and tested only against the versions listed here.

Plugin version	IDO Android	IDO iOS
0.0.5	com.ts.sdk.identityorchestration:1.0.34	identityOrchestration-ios-sdk 1.2.3

These are the versions the plugin declares and pins. The shared core SDK is resolved transitively by them and is not pinned here.

What this means in practice:

- **To move to a newer native SDK, move to a newer plugin version.** IDO versions are not configurable per integration.
- **If you already depend on IDO directly,** align on the version above or you will hit a resolution conflict. Contact Transmit Security if you need a combination this table does not cover.

The IDV SDK is NOT bundled — you add it, only if you want Modular IDV

The plugin does **not** declare the IDV native SDK. Apps that never run a Modular IDV journey therefore ship neither IDV nor its transitive AccountProtection dependency.

If you do want Modular IDV, you add IDV to your own app. See *Modular IDV → Adding the IDV native SDK* for the per-platform instructions and the **mandatory minimum version**.

	Required version
com.ts.sdk:identityverification (Android)	1.3.5 or higher
identityVerification-ios-sdk (iOS)	1.3.5 or higher

Native SDK Documentation

For platform-specific implementation details and advanced configuration, refer to the official Transmit Security native SDK documentation:

- **iOS WebAuthn SDK:** [Quick Start Guide](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_ios) (https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_ios)
- **Android WebAuthn SDK:** [Quick Start Guide](https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android) (https://developer.transmitsecurity.com/guides/webauthn/quick_start_sdk_android)

These guides provide detailed information about native SDK features, configuration options, and platform-specific implementation requirements.

Installation

1. Add Dependency

Add this to your `pubspec.yaml`:

```
dependencies:  
  flutter_ts_identity_orchestration:  
    git:  
      url:  
https://github.com/TransmitSecurity/flutter\_ts\_identity\_orchestration.git  
      ref: 0.0.3
```

Run:

```
flutter pub get
```

2. iOS Setup (If using camera and FaceID)

Add to your `ios/Runner/Info.plist`:

```
<key>NSCameraUsageDescription</key>  
<string>This app needs camera access for identity verification</string>  
<key>NSFaceIDUsageDescription</key>  
<string>This app uses Face ID for authentication</string>
```

Minimum iOS version in `ios/Podfile`:

```
platform :ios, '13.0'
```

3. Android Setup (If using camera)

Add to your android/app/src/main/AndroidManifest.xml:

```
<uses-permission android:name="android.permission.INTERNET" />
<uses-permission android:name="android.permission.CAMERA" />
```

Minimum SDK in android/app/build.gradle:

```
android {
    compileSdkVersion 34
    defaultConfig {
        minSdkVersion 21
        targetSdkVersion 34
    }
}
```

Integration

Maven Configuration

Add Maven repository to both the module and app-level build files:

Module level (android/build.gradle):

```
allprojects {
    repositories {
        google()
        mavenCentral()
        // Add Transmit Security Maven repository
        maven {
            url "https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/"
        }
    }
}
```

For Kotlin use:

```
maven {
    url = uri("https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/")
}
```

App level (android/app/build.gradle):

```
repositories {
    google()
    mavenCentral()
    // Add Transmit Security Maven repository
    maven {
        url "https://repo.transmitsecurity.io/repository/tspublic/"
    }
}
```

Android MainActivity Configuration

IMPORTANT: Make sure that your MainActivity inherits from FlutterFragmentActivity instead of FlutterActivity:

```
import io.flutter.embedding.android.FlutterFragmentActivity

class MainActivity: FlutterFragmentActivity() {
    // Your MainActivity code here
}
```

This is required for proper plugin functionality and native SDK integration.

Quick Start

1. Import the Package

```
import
'package:flutter_ts_identity_orchestration/flutter_ts_identity_orchestration.dart';
```

2. Initialize the SDK

```
class MyApp extends StatefulWidget {
    @override
    _MyAppState createState() => _MyAppState();
}

class _MyAppState extends State<MyApp> {
    final _orchestration = FlutterTsIdentityOrchestration();
    StreamSubscription<Map<String, dynamic>>? _subscription;

    @override
    void initState() {
        super.initState();
        _initializeSDK();
        _listenToResponses();
    }

    void _initializeSDK() async {
        try {
            // Option 1: Initialize with parameters
            await _orchestration.initialize(
                clientId: 'your-client-id',
                options: {
                    'serverPath': 'https://api.transmitsecurity.io',
                    'resource': null, // Optional: custom resource path
                    'pollingTimeout': null, // Optional: polling timeout in
milliseconds
                    'locale': 'en', // Optional: locale setting (default:
'en')
                },
        },
```

```

    );

    // Option 2: Initialize with config file (iOS: plist, Android: auto-
    detected)
    // await _orchestration.initializeSDK(configurationFile:
    'TransmitSecurity');

    print('SDK initialized successfully');
  } catch (e) {
    print('Failed to initialize SDK: $e');
  }
}

void _listenToResponses() {
  // CRITICAL: You MUST listen to journeyResponseStream to receive SDK
  responses
  // This stream provides real-time updates from the identity orchestration
  journey
  // including user interface steps, authentication results, and error
  notifications
  _subscription = _orchestration.journeyResponseStream.listen(
    (response) {
      print('Journey response: $response');
      _handleJourneyResponse(response);
    },
    onError: (error) {
      print('Journey error: $error');
    },
  );
}

@override
void dispose() {
  _subscription?.cancel();
  super.dispose();
}
}

```

⚠ **CRITICAL: Listen to Journey Responses**

Before starting ANY journey, you MUST set up response listening. This is not optional.

The `_listenToResponses()` method is **essential** for your app to function correctly:

- **What it does:** Creates a persistent connection to receive real-time updates from the Transmit Security SDK
- **When to call:** Immediately after SDK initialization, before starting any journeys
- **What it receives:** Authentication results, user interface instructions, error notifications, journey completion status
- **Without it:** Your app will start journeys but never receive responses - users will be stuck

```
// CORRECT: Listen first, then start journey
void initState() {
  super.initState();
  _initializeSDK();
  _listenToResponses(); // ← MANDATORY: Must be called before starting journeys
}

// INCORRECT: Starting journey without listening will result in no responses
void badExample() {
  _orchestration.startJourney(journeyId: 'test'); // Will hang indefinitely!
}
```

3. Start a Journey

```
void _startJourney() async {
  try {
    await _orchestration.startJourney(
      journeyId: 'your-journey-id',
      options: {
        'additionalParams': {'key': 'value'}, // Optional
        'flowId': 'custom-flow-id', // Optional
        'encryptionMode': true, // Optional: enable double encryption
      },
    );
    print('Journey started');
  } catch (e) {
    print('Failed to start journey: $e');
  }
}
```

4. Handle Journey Responses

```

void _handleJourneyResponse(Map<String, dynamic> response) {
  final bool isSuccess = response['success'] == true;

  if (!isSuccess) {
    print('Journey failed: ${response['error']}');
    return;
  }

  final responseData = response['response'];
  final String? stepId = responseData?['journeyStepId'];

  switch (stepId?.toLowerCase()) {
    case 'action:information':
      _showInformationScreen(responseData);
      break;
    case 'login_form':
      _showLoginForm(responseData);
      break;
    case 'action:rejection':
      _showRejectionDialog(responseData);
      break;
    default:
      print('Unhandled step: $stepId');
  }
}

```

Response fields

response['response'] contains:

Field	Type	Notes
journeyStepId	String?	The step to handle, or action:success / action:rejection
clientResponseOptions	Map?	Available client response options for this step
data	Map?	Step payload
token	String?	Proof of journey completion, present on success
code	String?	Backend token exchange code, present on success. iOS only — see below
errorData	Map?	errorCode and errorMessage when the step carries an error. errorCode is the SDK's snake_case value, e.g. idv_not_available

Platform note — code. Added in iOS native SDK 1.2.2, which fixed a defect where the backend token exchange code was parsed out of the server response and never reached the app. The Android SDK has no equivalent property at 1.0.34, so the key is **absent** from the response map on Android rather than present-and-null, and the plugin logs not implemented in SDK 1.0.34 once per plugin instance. Treat code as nullable and do not require it for a journey to be considered complete.

New step type — PIN code transaction signing. Both platforms now recognize this as a dedicated action type — iOS since native 1.2.2, Android since native 1.0.34. The raw server step id was already delivered to Dart unchanged on both platforms before this, so there is no behavior change for existing integrations; handle it from `journeyStepId` as you would any other step.

5. Submit Responses

```

void _submitResponse(String responseId, Map<String, dynamic>? data) async {
  try {
    final success = await _orchestration.submitClientResponse(
      responseId: responseId,
      data: data,
    );

    if (success) {
      print('Response submitted successfully');
    } else {
      print('Failed to submit response');
    }
  } catch (e) {
    print('Error submitting response: $e');
  }
}

// Example: Submit login credentials
void _submitLogin(String username, String password) {
  _submitResponse('clientInput', {
    'username': username,
    'password': password,
  });
}

// Example: Cancel current step
void _cancelStep() {
  _submitResponse('cancel', null);
}

```

API Reference

1. initializeSDK

Initialize SDK with configuration file.

```
Future<void> initializeSDK({String? configurationFile})
```

Example:

```
await _orchestration.initializeSDK(configurationFile:
'TransmitSecurity.plist');
```

This path reads credentials from platform configuration, so risk-data collection is enabled there rather than in code:

- **iOS** — add a top-level collectRiskData boolean to TransmitSecurity.plist. It is a **sibling** of credentials, not a key inside it:

```
<key>credentials</key>
<dict>...</dict>
<key>collectRiskData</key>
<true/>
```

- **Android** — add a bool resource:

```
<bool name="transmit_security_collect_risk_data">true</bool>
```

Both default to false when the key is absent. To set it in code instead, use [initialize](#) with collectRiskData in its options map.

2. initialize

Initialize SDK with parameters.

```
Future<void> initialize({
  required String clientId,
  required Map<String, dynamic> options,
})
```

Parameters:

- clientId (String): Your Transmit Security client identifier
- options (Map<String, dynamic>): Configuration options
 - serverPath (String): Server URL (default: 'https://api.transmitsecurity.io')
 - resource (String?): Optional custom resource path
 - pollingTimeout (int?): Optional polling timeout in milliseconds
 - locale (String?): Optional locale setting (default: 'en')
 - collectRiskData (bool?): Collect device data for fraud detection on journey startup (default: false). See [Risk data collection](#).

Example:

```
await _orchestration.initialize(
  clientId: 'your-client-id',
  options: {
    'serverPath': 'https://api.transmitsecurity.io',
    'resource': null,
    'pollingTimeout': 30000, // 30 seconds
    'locale': 'en',
    'collectRiskData': false,
  },
);
```

collectRiskData is read **only at initialization**. Changing it later has no effect until the SDK is initialized again.

Risk data collection

Setting collectRiskData: true makes the native SDK collect device data and attach it to the journey start request. Two consequences worth knowing before enabling it:

- **Android gains a transitive Play Services dependency.** The shared core SDK pulls in `com.google.android.gms:play-services-location` (plus `-base`, `-basement`, `-tasks`). If your app already depends on Play Services, check for a version conflict.
- **Geolocation is opt-in and must be granted by your app.** The SDK declares no location permission of its own and never prompts — Android's `TSGeolocationProvider` and iOS's location provider only *read* whether your app already holds authorization. If it does, location is included in the collected data; if not, collection proceeds without it and no error is surfaced.

Risk data works either way, so this is optional. To include location, declare and request it at the **app** level:

Android — `android/app/src/main/AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

These are dangerous permissions, so a manifest entry alone grants nothing on API 23+ — your app must also request them at runtime.

iOS — `Info.plist`, plus a `CLLocationManager.requestWhenInUseAuthorization()` call:

```
<key>NSLocationWhenInUseUsageDescription</key>
<string>Explain why your app includes location in risk data.</string>
```

iOS will not present the prompt without a usage description, and without a request the app never appears under Settings > Privacy > Location.

The example app in `example/` implements both, in `MainActivity.kt` and `AppDelegate.swift`.

If your app uses R8 or ProGuard, no extra configuration is needed — this plugin ships consumer rules covering the reflectively-serialized device-data classes.

3. startJourney

Start an identity verification journey.

```
Future<void> startJourney({
  required String journeyId,
  Map<String, dynamic>? options,
})
```

Parameters:

- `journeyId` (String): Journey identifier from the Admin Console

- options (Map<String, dynamic>?): Optional journey options
 - additionalParams (Map?): Additional parameters passed to the journey
 - flowId (String?): Flow identifier; auto-generated when omitted
 - encryptionMode (bool?): Enable double encryption
 - drsSessionToken (String?): DRS session token to associate the journey with a DRS session. Supported on both platforms — see below. An empty string is treated as omitted.

Example:

```
await _orchestration.startJourney(
  journeyId: 'registration-flow',
  options: {
    'additionalParams': {'userType': 'premium'},
    'encryptionMode': true,
    'drsSessionToken': 'drs-session-token',
  },
);
```

Platform note — drsSessionToken. Supported on iOS from native SDK 1.2.2 and on Android from native SDK 1.0.33. An empty string is treated as omitted on both platforms.

4. submitClientResponse

Submit response during journey flow.

```
Future<bool> submitClientResponse({
  required String responseId,
  Map<String, dynamic>? data,
})
```

Common Response IDs:

- 'clientInput' - Submit user input
- 'cancel' - Cancel current step
- 'fail' - Mark step as failed
- 'resend' - Request resend (e.g., OTP)

Example:

```
// Submit form data
await _orchestration.submitClientResponse(
    responseId: 'clientInput',
    data: {'email': 'user@example.com'},
);

// Cancel flow
await _orchestration.submitClientResponse(
    responseId: 'cancel',
    data: null,
);
```

⚠ Instructions Callback:

When calling `submitClientResponse`, `startMobileApproveJourney`, or `generateDebugPin`, the native SDK may trigger additional instructions through the event stream. These will be delivered via the `journeyResponseStream` with type `'instructions'`.

Handling Instructions:

```
_orchestration.journeyResponseStream.listen((response) {
    if (response['type'] == 'instructions') {
        final instructions = response['instructions'];
        print('Instructions received: $instructions');

        // Handle instructions data
        // Instructions format depends on the specific journey step
        // and may contain additional guidance or configuration
    }
});
```

5. generateDebugPin

Generate debug PIN for testing.

```
Future<String> generateDebugPin()
```

Example:

```
final pin = await _orchestration.generateDebugPin();
print('Debug PIN: $pin');
```

⚠ Instructions Callback:

`generateDebugPin` may also trigger instructions through the event stream with type `'instructions'` (implementation varies by platform).

6. startMobileApproveJourney

Start mobile approval journey.

```
Future<void> startMobileApproveJourney({
    required Map<String, dynamic> payload,
    required Map<String, dynamic> startJourneyOptions,
})
```

Example:

```
await _orchestration.startMobileApproveJourney(  
    payload: {'sessionId': 'session-123'},  
    startJourneyOptions: {'encrypted': true},  
);
```

⚠ Instructions Callback:

Similar to submitClientResponse, startMobileApproveJourney may also trigger instructions through the event stream with type 'instructions'.

7. setLoggingEnabled

Enable or disable SDK logging.

```
Future<bool> setLoggingEnabled(bool enabled)
```

Example:

```
await _orchestration.setLoggingEnabled(true); // Enable for debugging  
await _orchestration.setLoggingEnabled(false); // Disable for production
```

8. setPushToken

Set the push notification token for the SDK.

```
Future<void> setPushToken(String token)
```

Parameters:

- token (String): The push notification token from your push notification service

Example:

```
// Get token from Firebase/APNs and set it  
final token = await FirebaseMessaging.instance.getToken();  
if (token != null) {  
    await _orchestration.setPushToken(token);  
}
```

9. journeyResponseStream ⚠ MANDATORY

Stream for real-time journey events. **This is the ONLY way the SDK communicates with your app.**

```
Stream<Map<String, dynamic>> get journeyResponseStream
```

⚠ CRITICAL REQUIREMENT:

- You MUST listen to this stream BEFORE starting any journey
- Without this listener, your app will not receive any responses from the SDK
- All authentication results, UI instructions, and errors are delivered through this stream

Response Types You'll Receive:

- `action:information` - Display information to user
- `login_form` - Show login form
- `collect_totp_form` - Show TOTP/OTP input
- `action:rejection` - Authentication failed
- `success` - Journey completed successfully
- `instructions` - Native SDK instructions (triggered by `submitClientResponse`, `startMobileApproveJourney`, `generateDebugPin`)

Example:

```
// Set up listener BEFORE starting journeys
orchestration.journeyResponseStream.listen((response) {
  // Handle instructions from submitClientResponse
  if (response['type'] == 'instructions') {
    final instructions = response['instructions'];
    print('Instructions received: $instructions');
    // Process additional guidance or configuration
    return;
  }

  if (response['success'] == true) {
    final stepId = response['response']?['journeyStepId'];
    print('Current step: $stepId');

    // Handle different journey steps
    switch (stepId?.toLowerCase()) {
      case 'action:information':
        _showInformationScreen(response['response']);
        break;
      case 'login_form':
        _showLoginForm(response['response']);
        break;
      // ... handle other steps
    }
  } else {
    print('Journey error: ${response['error']}');
  }
});
```

Modular IDV (document and selfie acquisition)

Supported on **both platforms** — iOS from native SDK 1.2.3, Android from native SDK 1.0.34.

When a journey reaches a document or selfie acquisition step, the native SDK launches its own capture UI and continues the journey once the user finishes. **There is no plugin API to call:** the plugin registers the SDK's UI handler internally before each journey starts, and the flow is driven natively on both platforms.

What you must provide

Camera permission, or capture cannot start.

- **iOS** — NSCameraUsageDescription in ios/Runner/Info.plist (see *Installation → iOS Setup*). Without it iOS terminates the app when the camera is accessed.
- **Android** — android.permission.CAMERA in your manifest, plus a runtime request before the journey reaches an acquisition step. See the example app's MainActivity for the pattern.

The IDV native SDK, which the plugin does not bundle. See *Adding the IDV native SDK* below.

Adding the IDV native SDK

Modular IDV is opt-in. The plugin declares only IDO, so you add IDV to your own app — which also means apps that never run a Modular IDV journey carry neither IDV nor AccountProtection.

⚠ IdentityVerification 1.3.5 or higher is mandatory

This is a hard requirement, not a recommendation. **On iOS, an IDV version older than 1.3.5 will crash your app on launch** — for every user, whether or not they ever reach a Modular IDV step.

IdentityOrchestration weak-links the IDV framework but references its symbols as hard undefined. With no IDV present those symbols bind to null, the SDK's runtime class check fails cleanly, and you get `idv_not_available`. With an **older** IDV present, the framework loads and the class check *passes* — the `TSIdentityVerification` class predates Modular IDV — and then the missing `startDocumentAcquisition / startSelfieAcquisition` symbol is a `dyld symbol-not-found` error.

Android is more forgiving: its adapter reaches IDV by reflection and maps a missing method to `idv_not_available`, so an old IDV degrades exactly like no IDV. **Do not rely on that** — pin 1.3.5 or higher on both platforms.

Not adding IDV at all is always safe. Adding the wrong version is not.

Android — in `android/app/build.gradle` (or `.kts`):

```
dependencies {
    implementation "com.ts.sdk:identityverification:1.3.5"
}
```

Your repositories must include the Transmit Artifactory, which you already need for IDO:

```
maven { url 'https://transmit.jfrog.io/artifactory/transmit-security-gradle-release-local/' }
```

This pulls `com.ts.sdk:accountprotection` transitively.

iOS — add the Swift package to your Xcode project:

1. Open `ios/Runner.xcworkspace` (or `.xcworkspaceproj`) in Xcode.
2. **File → Add Package Dependencies...**
3. Enter `https://github.com/TransmitSecurity/identityVerification-ios-sdk`
4. Set the dependency rule to **Exact Version 1.3.5** (or Up to Next Major from 1.3.5).
5. Add the `IdentityVerification` product to the **Runner** target.

This pulls `accountprotection-ios-sdk` transitively. Confirm it worked by checking that `IdentityVerification.framework` appears under Frameworks in your built `.app`.

iOS is SPM-only. The Transmit IDO and IDV iOS SDKs are not published to CocoaPods, so Modular IDV cannot be added to a CocoaPods-based app. Contact Transmit Security if you need this.

If IDV is not linked and the server configures a Modular IDV step, nothing crashes and the build

succeeds — but **what the app is told differs by platform**, and on iOS it is currently told nothing at all.

Android

iOS

App launches, no crash

Journey reaching an acquisition step **rejected** with `idv_not_available`, in the `PlatformException's details['errorCode']` from `startJourney`

⚠ **no response at all — the journey hangs**

⚠ **Known issue on iOS.** With IDV absent, the native SDK logs `Failed to initialize modular IDV controller` and `Failed to start IDV selfie acquisition` (`IdoError 1008, "The SDK is not available"`) and **swallows both**. No error reaches the app and the journey never advances or terminates. Verified against IDO iOS 1.2.3.

If you ship an iOS app without IDV, do not rely on receiving an error for a Modular IDV step.

Apply your own timeout around a journey that could reach one, or ensure your journeys never configure Modular IDV for builds that omit the IDV SDK.

Android's behaviour is the intended one: a clean rejection the app can act on. See [Error codes](#) for where the code arrives.

Journey steps you will receive

journeyStepId	What to do
transmit_platform_document_acquisition	Nothing. The native capture UI is already driving the step; it reaches Dart only as a notification. Do not build a screen for it — it would race the native one.
transmit_platform_selfie_acquisition	Same. Note this step id is dual-purpose : it also carries the pre-existing face-authentication step, which is unaffected by Modular IDV.
transmit_platform_idv_recommendation	Poll. Resubmit an empty <code>clientInput</code> response each time this step is delivered.

The recommendation step

After acquisition, the server computes the IDV recommendation asynchronously and the journey parks on `transmit_platform_idv_recommendation` until it is ready. Poll by resubmitting:

```
case 'transmit_platform_idv_recommendation':
  // No delay, no attempt cap – the SDK and server handle pacing.
  await _orchestration.submitClientResponse(responseId: 'clientInput', data:
null);
```

Do **not** add client-side backoff or a retry limit. The journey advances on its own once the recommendation resolves. Keep one screen mounted across repeated deliveries rather than rebuilding it each round-trip.

Error codes

Modular IDV failures reach your app on more than one channel, depending on when they occur and on the platform. The `errorCode` **value** is the same everywhere; only the delivery differs.

- **The journey response stream** — either as `errorData.errorCode` inside a successful envelope (a step that carries an error), or as a top-level `errorCode` on a failure envelope.
- **The PlatformException from the call that failed** — `startJourney`, `startMobileApproveJourney` or `submitClientResponse`. Here `PlatformException.code` names the *operation* (`startJourneyError`), which every cause of that failure shares, so the native code travels in `details['errorCode']`.

⚠ **Handle both channels — which one carries a given failure differs by platform.** For the same condition (an acquisition step failing after the journey starts):

	Android	iOS
Where the failure arrives	the <code>PlatformException</code> from <code>startJourney</code>	the journey response stream, as <code>{'success': false, 'error': ..., 'errorCode': ...}</code>
This is a known asymmetry in the native SDKs' callback shapes, not a configuration choice. An app that handles only one channel will miss the failure on one platform — on Android it looks like a stream that goes quiet, on iOS like a call that succeeded. The <code>errorCode</code> value itself is identical on both; only the delivery differs.		

So there are three places an `errorCode` can appear, and a robust integration reads all three:

```
// 1. A step that carries an error, inside a successful envelope.
//    response['response']['errorData']['errorCode']

// 2. A journey failure on the stream (iOS).
tsID0.journeyResponseStream.listen((response) {
  if (response['success'] == false) {
    final code = response['errorCode'];           // e.g.
    'idv_camera_permission_required'
  }
});

// 3. A journey failure on the call (Android).
try {
  await tsID0.startJourney(journeyId: 'my_journey');
} on PlatformException catch (e) {
  final code = (e.details as Map?)?['errorCode'];
}
```

The values, identical on both platforms:

errorCode	Meaning
<code>idv_camera_permission_required</code>	Camera permission missing or denied
<code>idv_sdk_disabled</code>	IDV disabled by configuration
<code>idv_session_not_valid</code>	Session missing or invalid; a new one must be created server-side
<code>idv_initialization_error</code>	IDV failed to initialize (typically missing credentials)
<code>idv_not_initialized</code>	IDV not initialized
<code>idv_canceled</code>	User canceled the verification
<code>idv_internal_error</code>	Internal IDV error
<code>idv_not_available</code>	The IDV module is not available at runtime

If every acquisition step fails with `idv_not_available`, the IDV SDK is not linked into the build — the journey will be rejected server-side rather than crashing.

Upgrading from 0.0.4 or earlier? On Android these codes were previously emitted as the native enum's constant name (`IdvNotAvailable`) rather than the documented value (`idv_not_available`), and were **omitted entirely** from a failed call's details. Both are fixed in 0.0.5. If your app matches on the old PascalCase spelling, update it — see CHANGELOG for the full note.

Hooks: interjecting around acquisition

⚠ **Hooks require the IDV native SDK.** Do not enable them in an app that does not link IdentityVerification (see [Adding the IDV native SDK](#)). Without IDV, an acquisition step still fires the hook, but the resume call cannot open capture — it fails with `modularIdvHookError` and the step suspends with no way to complete it. In an app that does not link IDV, leave hooks off: the acquisition step is then rejected cleanly with `idv_not_available`.

By default acquisition proceeds automatically — the native capture UI opens, the user finishes, and the journey advances with no involvement from your Dart code. That is the behaviour you get without doing anything, and it does not change.

If you want to show your own screens around a step — instructions before capture, a review or consent screen after it — enable the hooks:

```
// Subscribe BEFORE enabling. See the warning below.
final subscription = tsID0.modularIdvHookStream.listen((event) async {
  // The journey is paused here until you resume it.
  final userAgreed = await showMyInstructions(event.acquisitionType);

  if (userAgreed) {
    // Continue along the SDK's default path: open capture (onBefore),
    // or submit the capture result (onAfter).
    await tsID0.proceedModularIdvStep(event.hookId);
  } else {
    // Or branch the journey instead of running the default path.
    await tsID0.submitModularIdvStep(hookId: event.hookId, responseId:
'cancel');
  }
});

await tsID0.setModularIdvHooks(onBefore: true, onAfter: true);
```

⚠ **An enabled hook stalls the journey until you resume it.** There is no timeout and no error. If you enable a hook and do not listen — or a code path through your listener returns without calling `proceedModularIdvStep` or `submitModularIdvStep` — the journey stops at that step permanently. Make sure every branch of your handler, including the failure branch, ends in exactly one resume call. This is the native hook's own contract, not a plugin limitation.

ModularIdvHookEvent

Field	Type	Notes
hookId	String	Pass back when resuming. Single-use — a second resume for the same id throws
hook	ModularIdvHookPoint	onBefore or onAfter
acquisitionType	ModularIdvAcquisitionType	document or selfie
acquisitionId	String?	Server-side session id, for correlation only — see below
responseOptions	List<ModularIdvResponseOption>	Options available for <code>submitModularIdvStep</code> ; may be empty

Notes that matter in practice:

- **Use hookId, not acquisitionId, to identify a step.** `acquisitionId` is shared by the `onBefore` and `onAfter` hook of the same step, repeats across the document and selfie steps of one journey, and may be null.
- **Hooks are toggled, not per-journey.** `setModularIdvHooks` applies from the next acquisition step. A step already paused in a hook keeps the behaviour it started with, and disabling

does not
resume it.

- **Starting a new journey abandons pending hooks.** Any hookId issued during a previous journey stops working, because the step it referred to no longer exists.
- **The IDV session token is not exposed.** It is a credential only the native SDK can act on, so it is deliberately not sent to Dart. `proceedModularIdvStep` is how you start capture.
- **Hook events do not appear on `journeyResponseStream`.** They arrive only on `modularIdvHookStream`, so enabling hooks cannot introduce unrecognized events into your existing journey handling.

The example app has a **Modular IDV Hooks** switch and a confirmation sheet showing the full round trip.

Error Handling

Common Error Patterns

```
try {  
  await _orchestration.startJourney(journeyId: 'test-journey');  
} on PlatformException catch (e) {  
  switch (e.code) {  
    case 'invalidArguments':  
      print('Invalid parameters provided');  
      break;  
    case 'sdkInitError':  
      print('SDK not initialized');  
      break;  
    case 'startJourneyError':  
      print('Failed to start journey: ${e.message}');  
      break;  
    default:  
      print('Unknown error: ${e.message}');  
  }  
} catch (e) {  
  print('Unexpected error: $e');  
}
```

Response Validation

```

void _validateResponse(Map<String, dynamic> response) {
  if (response['success'] != true) {
    final error = response['error'] ?? 'Unknown error';
    throw Exception('Journey failed: $error');
  }

  final responseData = response['response'];
  if (responseData == null) {
    throw Exception('Invalid response data');
  }
}

```

Best Practices

1. Initialization

- Initialize SDK once when app starts
- Handle initialization errors gracefully
- Use configuration files for production apps

2. Journey Management ⚠ CRITICAL

- **MANDATORY:** Always listen to journeyResponseStream BEFORE starting any journey
- The SDK communicates exclusively through this stream - without it, your app won't receive any responses
- Handle all possible journey steps in your response handler
- Implement proper loading states during journey execution

What happens without `_listenToResponses()`:

- Your app will start journeys but receive NO notifications when they complete
- Users will be stuck on loading screens with no way to proceed
- Authentication results, errors, and UI steps will be lost
- The journey will appear to "hang" indefinitely

3. Error Handling

- Implement comprehensive error handling for all API calls
- Provide user-friendly error messages
- Log errors for debugging (disable in production)

4. Security

- Never hardcode sensitive credentials in your app
- Use secure storage for configuration data
- Enable logging only during development

5. Performance

- Cancel stream subscriptions when not needed
- Avoid multiple simultaneous journey executions
- Cache SDK initialization state

Troubleshooting

iOS Issues

Problem: Build fails with missing frameworks

Solution: Ensure iOS deployment target is 13.0+

Problem: Camera/Location permissions denied

Solution: Add proper usage descriptions in Info.plist

Android Issues

Problem: Network security errors

Solution: Add network security config for HTTP domains (if needed)

Problem: Proguard issues in release builds

Solution: Add proguard rules for Transmit Security SDK

Common Issues

Problem: "SDK not initialized" error

Solution: Call `initialize()` or `initializeSDK()` before other operations

Problem: Journey responses not received / Journey appears to hang

Solution: You MUST subscribe to `journeyResponseStream` before starting journeys

Symptoms:

- Journey starts but never completes
- Loading screens that never end
- No error messages or success notifications
- App appears frozen during authentication

Fix:

```
// Correct order:
_listenToResponses(); // 1. Set up listener FIRST
await _startJourney(); // 2. Then start journey

// Wrong order:
await _startJourney(); // Journey will hang!
_listenToResponses(); // Too late - responses already lost
```

Problem: Invalid journey step handling

Solution: Implement handlers for all expected journey steps

Platform Differences

Login Form Response ID: The response ID for login form submissions differs between platforms:

```
import 'dart:io' show Platform;

// Use platform-specific response ID
final responseId = (Platform.isAndroid) ? 'password' : 'Password';

await orchestration.submitClientResponse(
  responseId: responseId,
  data: {
    'username': username,
    'password': password,
    'email': email,
  },
);
```

Example App Structure

```
lib/
├─ main.dart                # App initialization
├─ services/
│   └─ orchestration_service.dart # SDK wrapper
├─ screens/
│   ├── login_screen.dart    # Login form
│   ├── info_screen.dart     # Information display
│   └─ pin_screen.dart       # PIN input
└─ utils/
    └─ journey_handler.dart  # Journey response logic
```

This guide covers everything needed to integrate the Flutter TS Identity Orchestration plugin. For additional support, refer to the official Transmit Security documentation.